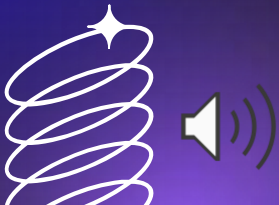




Planificadores de CPU con IA en Linux

Carlos Fernando Carballido y Raúl Haziél Sánchez
Sistemas Operativos, FI UNAM · 5 de octubre de 2026





Lo que vamos a ver



01

La IA como recurso

Qué es una NPU y quién
decide cómo se usa

02

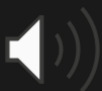
`sched_ext`

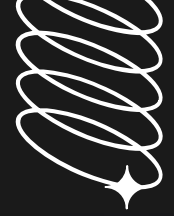
Cambiar el planificador sin
recompilar el kernel

03

Planificar con IA

Qué ya funciona y qué sigue
abierto

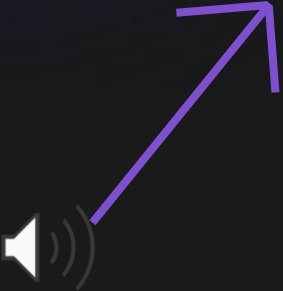




01

La IA como recurso

Antes de llegar al planificador





¿Qué es una NPU?

Es un chip hecho para los cálculos de redes neuronales. Para ese trabajo es más rápido que la CPU y gasta mucha menos batería.

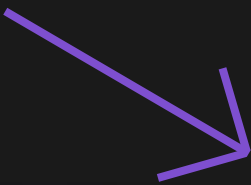
- Cada tarea de IA puede ir a la CPU, a la GPU o a la NPU, y el SO tiene que escoger
- Es el mismo tipo de decisión que toma al planificar procesos
- El planificador que traen las NPU de fábrica no contempla tiempo real (Yao et al., 2025)





Esto no empezó con ChatGPT





Cómo lo hace cada sistema



	Windows	macOS	Linux	Android
¿IA por defecto?	Sí	Sí	No	Sí
Hardware	NPU	Neural Engine	La del equipo, si trae	NPU / Tensor
Ejemplo	Recall, Auto SR	Apple Intelligence	Drivers en accel/	HDR+, Adaptive Battery
Enfoque	Parte central del producto	Privacidad, local primero	Opcional, decide el usuario	Mejorar el uso diario





02

sched_ext

El planificador como programa cargable





Antes y después de sched_ext

≤ 6.11

Antes

Linux usaba un planificador para todo: CFS y, desde la 6.6, EEVDF. Probar otra política implicaba parchar el kernel, recompilar y reiniciar.

6.12+

Desde Linux 6.12

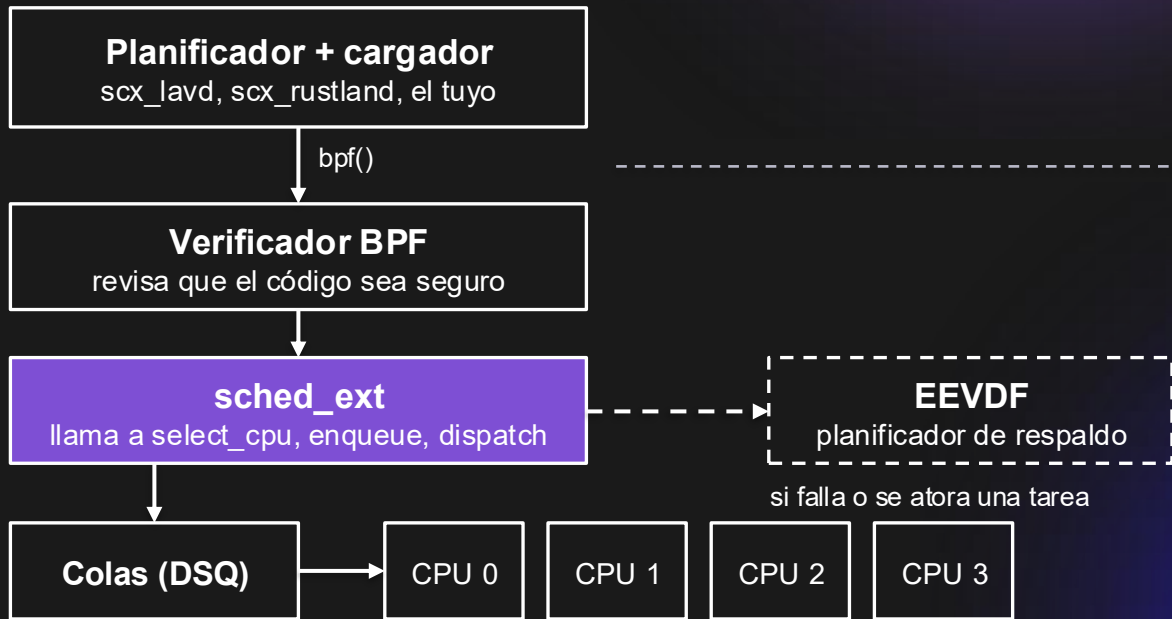
La política se escribe en BPF y se carga con el sistema corriendo. Si falla o deja tareas sin correr, el kernel la quita y vuelve al planificador normal.



Dónde vive cada parte

Espacio de usuario

Kernel





Cómo decide un planificador BPF

1

select_cpu

Cuando una tarea despierta, escoge en qué CPU conviene ponerla.

2

enqueue

Recibe la tarea lista para correr y la mete en una cola (DSQ).

3

dispatch

Cuando una CPU se queda sin trabajo, le pasa tareas desde las colas.

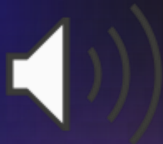


Un planificador en pocas líneas

Así se ve `scx_simple`, el ejemplo que viene con el kernel. Todas las tareas van a una sola cola y las CPUs las van tomando en orden.

```
$ sudo scx_simple -f  
$ cat /sys/kernel/sched_ext/root/ops
```

```
/* scx_simple.bpf.c, resumido */  
void BPF_STRUCT_OPS(simple_enqueue,  
    struct task_struct *p, u64 enq_flags)  
{  
    /* todas a la misma cola */  
    scx_bpf_dsq_insert(p, SHARED_DSQ,  
        SCX_SLICE_DFL, enq_flags);  
}  
  
void BPF_STRUCT_OPS(simple_dispatch,  
    s32 cpu, struct task_struct *prev)  
{  
    scx_bpf_dsq_move_to_local(SHARED_DSQ);  
}
```



Planificadores que ya se usan

scx_lavd

Lo hizo Igalia para Valve, pensando en el Steam Deck. Detecta qué tareas son sensibles a la latencia. Meta ya lo usa en servidores.

scx_bpfland

De Andrea Righi. Da prioridad a las tareas interactivas y toma todas sus decisiones dentro de BPF.

scx_rustland

Del mismo autor. Decide en espacio de usuario, en Rust. Es más flexible, pero cada viaje entre kernel y usuario cuesta.

scx_layered

De Meta. Separa las tareas en capas y le pone reglas distintas a cada una.

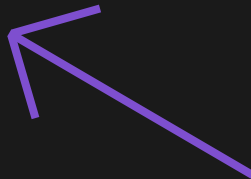
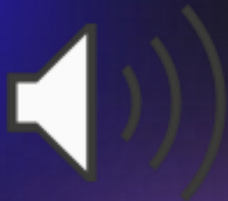


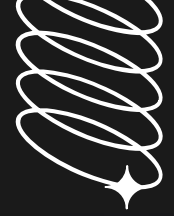
De una consola a los servidores de Meta

scx_lavd nació para que los juegos del Steam Deck respondieran mejor. En la Linux Plumbers Conference 2025, Meta contó que lo está usando como planificador por defecto en sus servidores.



Foto: Liam Dawe / GamingOnLinux, CC BY-SA 4.0, Wikimedia Commons

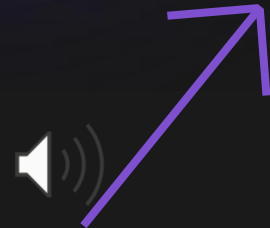




03

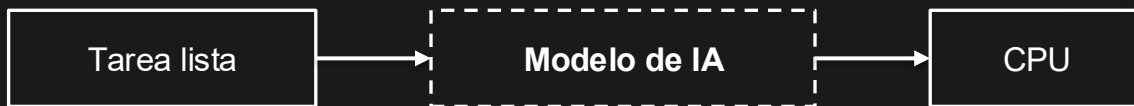
¿Y dónde entra la IA?

Dos formas de meter un modelo



Dos formas de usar IA

1. El modelo decide cada vez

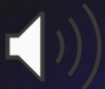


Cada cambio de tarea espera la inferencia. Muy lento.

2. El modelo ajusta desde afuera



Cada decisión corre a velocidad de BPF. Así funciona SchedCP (Zheng et al., 2025).





1.79x

mejora máxima frente a EEVDF al compilar el kernel

13x

menos costo que un agente LLM sin ese marco

40 TOPS

de NPU pide Microsoft para un Copilot+ PC

Fuentes: Zheng et al. (2025); Microsoft (s.f.)





Casos que nos llamaron la atención

LinnOS

OSDI 2020. Una red neuronal chiquita dentro del kernel predice si una lectura al SSD va a tardar. Cada inferencia toma de 4 a 6 μ s.

ghOSt

Google, SOSP 2021. Antes de sched_ext ya mandaba las decisiones de planificación a espacio de usuario. Lo probaron con Google Search.

Terraria

Demo de Andrea Righi. Con el kernel compilándose de fondo, el juego da más FPS con scx_rustland que con EEVDF.

Debian

Activó sched_ext en su kernel en abril de 2025 (bug #1102639), con la versión 6.14 de experimental.

Ubuntu 25.04

Su kernel 6.12 trae sched_ext activado. Se pueden probar planificadores sin compilar nada.

scxtop

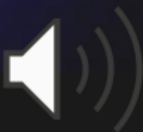
Un top para sched_ext. Exporta trazas a Perfetto y tiene un modo MCP para que una IA lea las métricas.





“Un truco publicitario de IA”

Canonical, sobre lo que no quiere para Ubuntu (Larabel, 2026)





Lo que todavía falta

Meter IA al sistema operativo también trae problemas:

- Un modelo grande es lento para el ritmo del planificador
- Si una IA genera código que corre en el kernel, hay que verificarlo
- Recall podía capturar contraseñas visibles en pantalla
- Las NPU todavía no garantizan tiempos de respuesta





Para cerrar

Desde el sistema operativo, la IA es un recurso más que hay que repartir y proteger, como la memoria o la CPU. sched_ext hace fácil probar planificadores nuevos, y por ahora la IA rinde más eligiendo la política que tomando cada decisión.



Referencias

Menychtas, K., Shen, K. y Scott, M. L. (2014). Disengaged scheduling for fair, protected access to fast computational accelerators. ASPLOS '14, 301-316. ACM.

Panneerselvam, S. y Swift, M. M. (2012). Operating systems should manage accelerators. HotPar '12. USENIX.

Yao, Y. et al. (2025). Workload-aware performance model based soft preemptive real-time scheduling for neural processing units. IEEE TPDS.

Zheng, Y., Hu, Y., Zhang, W. y Quinn, A. (2025). Towards Agentic OS: An LLM agent framework for Linux schedulers. arXiv:2509.01245.

Hao, M. et al. (2020). LinnOS: Predictability on unpredictable flash storage with a light neural network. OSDI '20. USENIX.

Humphries, J. T. et al. (2021). ghOST: Fast & flexible user-space delegation of Linux scheduling. SOSP '21. ACM.

The Linux Kernel Organization. (s.f.). Extensible Scheduler Class. kernel.org/doc/html/latest/scheduler/sched-ext.html

Righi, A. (2024). Crafting a Linux kernel scheduler that runs in user-space using Rust. Linux Plumbers Conference.

Larabel, M. (2025). Meta is using the Linux scheduler designed for Valve's Steam Deck on its servers. Phoronix.

Larabel, M. (2026). Linux desktops embrace AI on their own terms. WebProNews.

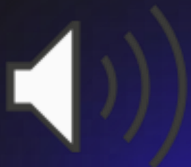
Android Developers Blog. (2018). A smarter smartphone, with machine learning at the core. Google.

Microsoft. (s.f.). Develop AI applications for Copilot+ PCs. Microsoft Learn.

Apple. (s.f.). Apple Intelligence and privacy on Mac. Soporte de Apple.

Righi, A. (2025). Ubuntu 25.04 is now sched_ext ready. arighi.blogspot.com

Debian. (2025). Bug #1102639: linux: Please enable CONFIG_SCHED_CLASS_EXT=y.





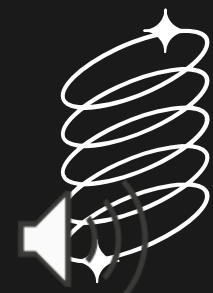
¡Gracias!

¿PREGUNTAS?

Carlos Fernando Carballido

Raúl Haziel Sánchez

Sistemas Operativos 2027-1



CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)

